To T or not to T: A story about small-angle rotations, fault-tolerant quantum computing, and the transversal STAR architecture. YQuantum 2026 — QuEra Challenge

The $\hat{O}$perators

Why can single-qubit gates be so difficult to implement fault-tolerantly? By exploring the concept of universal quantum computing, the difficulties arising from the Eastin-Knill theorem, clever techniques like magic state distillation and injection as well as the transversal STAR architecture, we learn how implementing a seemingly trivial 1-qubit rotation is one of the most profound bottlenecks on the road to fault-tolerant quantum computing (FTQC).

I. INTRODUCTION

At first glance, it appears implementing a single-qubit gate should obviously be easier than implementing a two-qubit gate. After all, there are fewer physical interfaces required to address one single qubit than for addressing two qubits at the same time. Right? While this is indeed the case for single-qubit Pauli gates and Clifford operations in general, the implementation of coherent rotations poses a major bottleneck in modern quantum computing. What makes implementing these non-Clifford gates so much harder than their Pauli counterparts? It all boils down to two fundamental problems:

1. Quantum computing platforms cannot implement any arbitrary gate at the hardware level. They are resctricted to some set of elementary operations, called a *universal gate set*. Whereas some specific single-qubit gates and the two-qubit CNOT gate can usually be readily implemented without big problems, arbitrary unitary rotations need to be approximated by a synthesizing circuit. In fact, the smaller the rotation, the more complex the circuit required to approximate the gate.

2. It is not possible to implement all operations of a universal gate set in an easy and fault-tolerant way at the logical level (we will specify in the next section what that exactly means). Therefore, if we want to implement the complex circuit needed for synthesizing a unitary rotation at the logical level, we need to resort to some clever tricks, but which come at a cost, further increasing the complexity of implementing just one single-qubit gate.

In the following pages, we are going to walk the reader through the technical difficulties of implementing small-angle rotations, introducing some of the sophisticated protocols and architecture schemes that have been developed recently, paving the road to large-scale fault-tolerant quantum computing (FTQC). We are going to present how analog rotations can be synthesized by a discrete set of universal gates, how non-Clifford gates can be implemented at the logical level by *magic state distillation* and *injection*, and how the *STAR architecture* enables a more efficient alternative.

II. SYNTHESIZING ANALOG ROTATIONS

In classical computation, every possible circuit can be composed of nothing but NAND gates. In other words, any logical operation one could think of can be synthesized by a circuit of NAND gates. In quantum computing, there exists a similar so-called *universal gate set*, one possible realization of which is given by $\{H, S, \mathrm{CNOT}, T\}$. In theory, it is therefore enough for a quantum computer to implement these four gates, as any other unitary operation can be approximated by a combination of the universal gates. In what follows, we will focus on the family of qubit rotations

$$R_z(\pi/2^k) = \begin{pmatrix} e^{-i\pi/2^{k+1}} & 0 \\ 0 & e^{i\pi/2^{k+1}} \end{pmatrix} \sim \begin{pmatrix} 1 & 0 \\ 0 & e^{2i\pi/2^{k+1}} \end{pmatrix},$$

where we used the fact that a quantum state is unique only up to a global phase factor.

A. Arbitrary Rotations from Clifford + T

$$\begin{aligned} k=0: &\quad R_z(\pi) = \begin{pmatrix} e^{-i\pi/2} & 0 \\ 0 & e^{i\pi/2} \end{pmatrix} = \begin{pmatrix} -i & 0 \\ 0 & i \end{pmatrix} = Z = S^2 = T^4 \\ k=1: &\quad R_z(\pi/2) = \begin{pmatrix} e^{-i\pi/4} & 0 \\ 0 & e^{i\pi/4} \end{pmatrix} = S = T^2 \\ k=2: &\quad R_z(\pi/4) = \begin{pmatrix} e^{-i\pi/8} & 0 \\ 0 & e^{i\pi/8} \end{pmatrix} \end{aligned}$$

First of all, it is worth noting that for $k = 0, 1, 2$, the gate operation $R_z(\pi/2^k)$ directly corresponds to the gates Z, S, and T, respectively (up to global phase). Note that the T gate effectively generates the finite cyclic group of rotations of 8 divisions on the x-y-plane, as illustrated in Figure 1. For $k \geq 3$, however, we must escape these rotations using the Hadamard gate. Thus we consider the commutation of T and H gates.

The Solovay-Kitaev theorem [1] guarantees that such approximations are possible to arbitrary position, and modern procedures like the Ross-Selinger algorithm [2] offer highly optimized automated synthesis with only

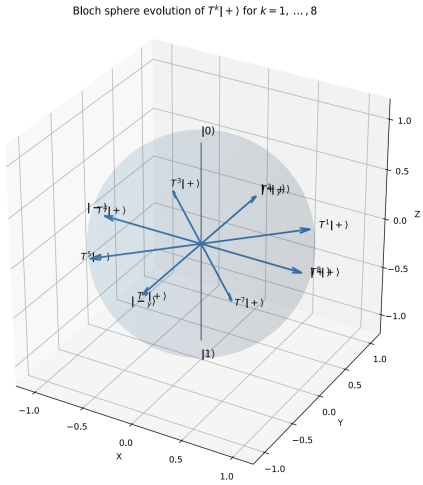

FIG. 1. Visualization of rotations generated by the T-gate acting on the $|+\rangle$ state.

logarithmic overhead of T-gates.

However, if we explore the synthesis ourselves, we realize that T and HTH correspond to our "building blocks" of Z and X rotations, respectively, together which can make Y rotations: $T(HTH)T^{-1}(HTH)^{-1}$. Visually, one can imagine the state vector on the Bloch sphere iteratively circling around the z-axis. The repeated application of H- and T-gates slowly drives the state away from the poles and towards the equator, eventually settling on the point with the specified target angle.

To quantitatively evaluate how well a synthesized circuit V approximates the ideal unitary gate U, we use the distance metric

$$d(U, V) = \sqrt{1 - \frac{|\mathrm{Tr}(U^\dagger V)|}{2}}$$

While the above methods guarantee a logarithmic overhead, it is also useful to explore naive solutions to gain a stronger intuition for the synthesis problem. We utilized a brute force approach with several iterations. The first iteration was a naive generation of all N-gate sequences followed by a calculation of each sequences distance from ideal, while slow this method did succeed at yielding an approximation for $k = 3$. In the second iteration we introduced a simple pre-selection optimization to only generate "useful" sequences (e.g. two H gates in a row is never useful), this allowed for faster sequence generation but ultimately left $k \geq 4$ unsolved. At this point, we inserted composite gates (e.g. an A gate which was equivalent to TH) into the gate set. This effectively allowed us to search for longer and ideally better approximations, since each composite gate represented more than one actual native gate, leading us to find reasonably close solutions for $0 \leq k \leq 5$. Lastly, we implemented a memoization strategy to lower the overall runtime of our strategy.

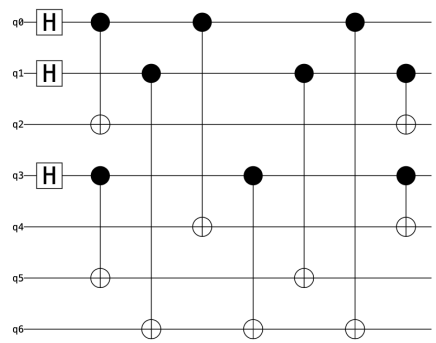

FIG. 2. Encoding circuit for the [[7,1,3]] Steane code.

B. Quantum Error Correction

So far, we have assumed that T-gates can be applied freely and flawlessly. In a fault-tolerant regime, however, this assumption is far from true. If we want to be able to correct errors, we cannot perform operations at the physical level, but need to encode our qubits using quantum error-correcting codes and perform computation at the *logical level*. More specifically, each logical qubit is encoded by a certain number of physical qubits, in a way that enables the detection and correction of errors that might affect the physical qubits. One out of endless possibilities to do this is the [[7,1,3]] *Steane code*, which encodes one logical qubit using 7 physical ones.

But what does this have to do with T-gates not being naively applicable anymore? The problem boils down to how we apply our quantum gates at the logical level. The most obvious way to apply a certain single-qubit gate to a logical qubit encoded by n physical qubits is to simply apply that gate to each and every of the n physical qubits individually. However, it is by no means clear that these individual operations at the physical level really translate into the desired effect of applying the logical operation to the logical qubit. If, given some quantum error-correcting code (QECC), this property holds for a certain gate, then we say that this gate is *transversal* in the code under consideration. Luckily, for the [[7,1,3]] Steane code, the Clifford gates $\{H, S, \mathrm{CNOT}\}$ from our universal gate set are transversal. However, there is one catch! It just so happens that nature forbids us to construct a complete universal gate set purely out of transversal gates; a result which is known as the *Eastin-Knill theorem*.

Consequently, this means that given the transversal H, S, and CNOT, the Steane code does not admit a transversal T-gate. We have a big problem: Synthesizing small-angle rotations without T-gates does not work well,

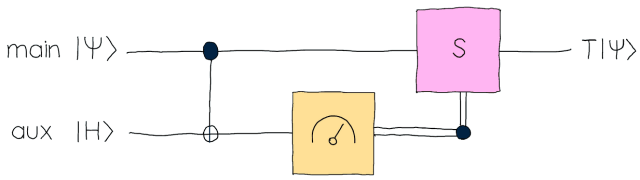

FIG. 3. Schematic Magic State Injection Circuit With Feedforward Operation. Source: https://pennylane.ai/qml/glossary/

but implementing T-gates at the logical level is not possible either (at least not in an easy way through transversality). It appears we need some magic to resolve this obstacle... and this is exactly what we are going to do!

C. Magic State Injection

The *magic* relies in the trick to not apply T-gates directly onto the logical qubits, but to prepare auxiliary qubits in so-called *magic states*. These states, when injected onto a logical qubit using a circuit somewhat similar to quantum teleportation, yield the same result as applying a logical T-gate to the encoded state.

Initial preparations of these magic states are inherently noisy, which is why we must make use of sophisticated routines like magic state distillation or magic state cutivation to create high-fidelity magic states using Clifford operations only. We are not going to describe these processes in detail, but it should be noted that they again drastically increase the overhead needed to perform the single-qubit rotation. For the sake of simplicity, we thus assume we have access to perfectly perpared magic states

$$|T\rangle = \frac{1}{\sqrt{2}}\Big(|0\rangle + e^{i\pi/4}\,|1\rangle\Big).$$

The *injection* of the effect of T-gates onto the logical qubits is achieved by using a simple teleportation-like circuit: A logical qubit $|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle$ and the magic state $|T\rangle$ are entangled via a CNOT-gate, which yields

$$\frac{1}{\sqrt{2}}\Big(\alpha\,|0\rangle + \beta e^{i\pi/4}\,|1\rangle\Big)\,|0\rangle + \frac{1}{\sqrt{2}}\Big(\alpha e^{i\pi/4}\,|0\rangle + \beta\,|1\rangle\Big)\,|1\rangle\,.$$

If now the auxiliary qubit is measured and a S-gate is applyied to the main qubit conditioned on the measurement outcome (applied if we masure $|0\rangle$, not applied if we measure $|1\rangle$), then we finally obtain the state $T\,|\psi\rangle$. Alternatively, instead of this mid-circuit measurement and feed-forward operation, we can also simply perform post-selection to filter the results.

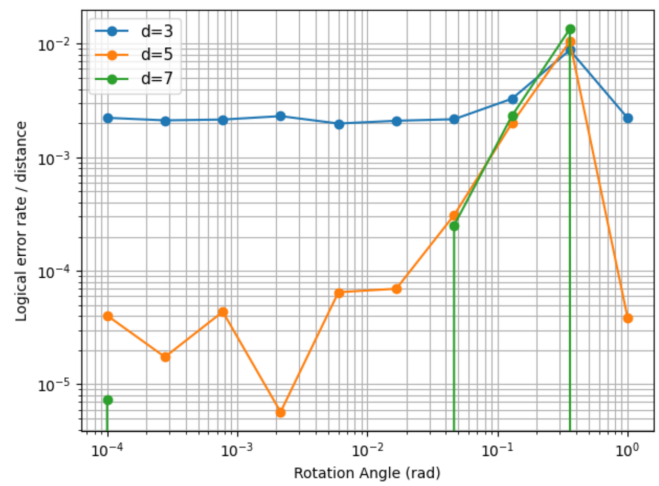

FIG. 4. Logical error rate (normalized to distance) versus the rotation angle for different code distances d, simulated under a circuit-level noise model. We see that the logical error rate plateaus towards smaller angles (there are some artifacts due to low shot rate).

III. TRANSVERSAL STAR ARCHITECTURE

Although effective, the production of these magic states needed to inject T-gates on the logical level entails massive resource overheads. A new approach trying to reduce this bottleneck is the transversal space-time efficient analog rotation (STAR) architecture [3, 4].

A. Small-Angle Magic State Injection

The main idea is to directly prepare small-angle magic states on auxiliary qubits and inject them using a post-selection protocol, instead of preparing and injecting T-states and using them to synthesize the small-angle rotation through deep Clifford+T circuits. By applying physical rotations transversally across the code block (something the neutral atom architecture is perfectly suited for due to its reconfigurable physical connectivity), the transversal STAR architecture achieves a logical error rate that scales linearly with the rotation angle θ, i.e.,

$$p_L(\theta) \sim |\theta| p_{ph},$$

where p_{ph} is the physical error rate. This property, which is also illustrated in Figure 4, makes the implementation of small-angle continuous rotations extremely efficient compared to discrete synthesis (as can be seen from the figure, the logical error rate plateaus towards smaller angles, which makes the STAR architecture perfectly well suited in that regime). So, how does it work?

1. We start by preparing the $n = d^2$ data qubits into a logical $|+\rangle$ state.

2. For a distance d surface code, we apply a physical z-rotation to d specific data qubits, which, when combined, results in an overall rotation on the logical level. The physical rotation angle θ_{ph} and the logical rotation angle θ_L are related through

$$\theta_{ph} = 2\arctan\left(\tan\left(\theta_L/2\right)^{1/d}\right).$$

3. Perform two rounds of X and Z syndrome extraction and post-select on perfect stabilizer measurements. This is necessary since applying continuous, non-Clifford rotations introduces coherent errors that the code might not be able to correct.

4. After post-selection, the circuit performs d more rounds of syndrome extraction. This ensures that all coherent errors injected by the non-Clifford rotation are fully projected back onto the codespace.

In the simulation, we finally applied an exact logical reverse rotation $R_z(-\theta_L)$ to undo the initial rotation. While obviously not possible in a real quantum computation, this final step enables the simulator to detect logical errors that occurred during the process, from which the logical error rate can be estimated.

B. Why Even Care About Small-Angle Rotations?

After all this effort to implement a single one-qubit gate, one might wonder, why we would even need such arbitrary small-angle rotation gates $R_z(\pi/2^k)$. Why not simply design quantum algorithms using only the Clifford+T universal gate set? The reason is that the most promising quantum algorithms fundamentally rely on continuous phase spaces. Restricting ourselves to large, discrete rotation angles (S- and T-gates) would drastically limit the computational power of the quantum computer. Two of the most prominent examples where small-angle rotations are crucial are the Quantum Fourier Transform (QFT) and quantum simulation.

1. The Quantum Fourier Transform, the main algorithmic engine behind Shor's algorithm for prime factorization, utilizes heavy entanglement of the logical qubits by controlled-phase rotations, where the rotation angle is $\pi/2^{k-1}$. Factoring an RSA key requires thousands of logical memory qubits, which implies the QFT would necessitate rotations with exponentially small angles such as $\pi/2^{1000}$.

2. Beyond breaking cryptography, a prominent application of quantum computing lies in quantum simulation. That is, the simulation of complex physical many-body systems. For simulating a certain system with Hamiltonian $\hat{H}$, one must implement the corresponding time-evolution operator $U(t) = e^{-i\hat{H}t}$, which is done by breaking the total simulation time t into very short timesteps Δt, a process called Trotterization. By decomposing the Hamiltonian into a sum of Pauli operators, it can be seen that this iterative simulation of small time steps corresponds exactly to the application of small-angle Pauli rotations.

Disclaimer: *We utilized LLMs for linguistic refinement and proofreading, and for making some graphics. The core framework, original code, and simulations were developed exclusively by the human authors.*

[1] C. M. Dawson and M. A. Nielsen, The Solovay-Kitaev algorithm, arXiv:quant-ph/0505030 (2005).

[2] N. J. Ross and P. Selinger, Optimal ancilla-free clifford+T approximation of z-rotations, arXiv:1403.2975 (2014).

[3] Y. Akahoshi, K. Maruyama, H. Oshima, S. Sato, and K. Fujii, Partially fault-tolerant quantum computing architecture with error-corrected clifford gates and space-time efficient analog rotations, arXiv:2303.13181 (2023).

[4] R. Toshio, Y. Akahosho, J. Fujisaki, H. Oshima, S. Sato, and K. Fujii, Partially fault-tolerant quantum computing architecture with error-corrected clifford gates and space-time efficient analog rotations, arXiv:2408.14848 (2024).

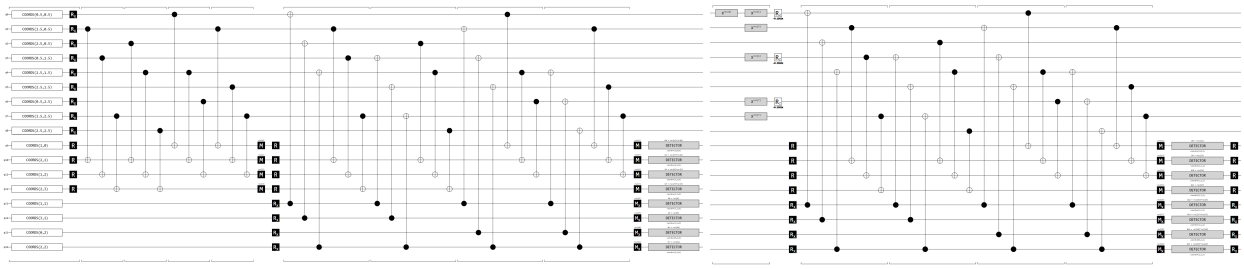

FIG. 5. First few timesteps for the STAR distance-3 circuit (without depolarizing noise for better visualization). From left to right, we see the state initialization, a couple of CNOTs followed by measurements for the syndrome extraction, three physical rotations, and finally the first out of three rounds of additional syndrome extraction.